

Tokenization in Large Language Models

Tokenization is the critical first step in an LLM's workflow. It involves breaking raw text into smaller, meaningful units called **tokens**. The LLM operates exclusively on these tokens, not on the raw characters or words themselves.

1. The Need for Tokenization

The core problem for an LLM is that its vocabulary (the set of all words it knows) is fixed. Tokenization serves three main purposes:

1. **Numerical Conversion:** LLMs process numerical vectors. Tokens convert unstructured text into a sequence of discrete integers (IDs) that can be mapped to embedding vectors for mathematical processing.
2. **Vocabulary Management:** A truly word-based vocabulary would require hundreds of thousands of entries to cover every word, proper noun, and variant. Tokenization keeps the vocabulary size manageable (even when the LLM covers multiple languages).
3. **Handling Unknown Words (Out-of-Vocabulary):** By breaking words into sub-units, the model can still assign meaning to rare or new words (like "untokenizeable") even if the full word was never seen in training.

2. How Tokenization is Performed

A tokenizer is a function that maps a string of text to a list of integers.

Key Parameters of a Tokenizer:

- **Vocabulary Size (V):** The total number of unique tokens the model can recognize. This is a hyper-parameter, typically ranging from 30,000 **to over** 100,000 for modern LLMs. A larger vocabulary can represent common words with a single token but increases model size and training complexity.
- **Maximum Context Length (L):** The maximum number of tokens the model can process simultaneously (e.g., 4096, 8192, or more). The input text is truncated if it exceeds this limit.

The Objective Function (Building a Good Tokenizer):

The goal is to design a tokenizer that produces a short sequence of tokens that is still semantically rich. Specifically, a good tokenizer must:

$$\text{Minimize } \left(\sum_{i=1}^N \text{Tokens}(\text{Sentence}_i) \right) \quad \text{subject to} \quad \text{VocabularySize} \leq V$$

This means: **Use the fewest possible tokens to represent the input text, given a fixed vocabulary size.** Shorter token sequences mean faster inference, reduced computational cost, and a larger effective context window (the LLM can "see" more of the original text).

An additional constraint on the tokenizer method is that it must be invertible and deterministic:

```
Detokenize(Tokenize(string)) == string  
Tokenize(Detokenize(list-of-tokens)) == list-of-tokens
```

3. The Necessity of Sub-Word Tokenization

Traditional tokenizers (like WordPiece or BPE) break the text into units that are often smaller than a full word but larger than a single character. This is necessary because:

1. **Inflections/Variations:** The word "learning," "learns," and "learned" might share a common root. Sub-word units allow the model to learn a single representation for the root "learn" and keep the semantic similarity between these words.
2. **Composites:** Compound words or technical jargon (e.g., "microprocessor") can be broken down into known components ("micro," "processor").
3. **Out-of-Vocabulary (OOV) Words:** If the model hasn't seen the word "Kubernetes," it can break it into known sub-words like "Kuber," "net," and "es," allowing the LLM to process it instead of replacing it with an `<UNK>` (unknown) token.

4. Byte Pair Encoding (BPE)

Byte Pair Encoding is the most popular sub-word tokenization algorithm used in models like GPT-3.

The BPE Process:

BPE is an iterative data compression algorithm that learns a vocabulary based on the training corpus.

1. **Initialization:** Start with a vocabulary of every unique character in the training corpus.
2. **Iterative Merging:** Repeatedly find the most frequent adjacent pair of bytes/characters and replace all occurrences of that pair with a new, single token.
3. **Stop:** The process repeats until the desired **Vocabulary Size** is reached, or no frequent pairs remain.

Concrete BPE Example

Consider a corpus with these words and frequencies:

Word	Frequency
l o w e r	5
n e w e s t	2
w i d e s t	3

Initial Vocabulary: {'l', 'o', 'w', 'e', 'r', 'n', 's', 't', 'i', 'd'}

Step 1: Merge e and s (most frequent pair)

- Corpus becomes: l o w e r (5), n e w e s t (2), w i d e s t (3)
- **New Token:** 'es'
- Vocabulary size ↑ 1

Step 2: Merge e and st (next most frequent pair)

- Corpus becomes: l o w e r (5), n e w e s t (2), w i d e s t (3)
- **New Token:** 'est'
- Vocabulary size ↑ 1

Tokenizing a new word, 'lowering':

The tokenizer would find the best way to break down the word using the learned tokens.

Input: lowering → Tokens: ['l', 'o', 'w', 'er', 'i', 'n', 'g']

(Assuming 'er' was also a frequent pair learned earlier).

In practice, before this mechanism is applied, the input text is split into space-separated character sequences (using regexps), and the resulting sequences are processed with the algorithm described above.

Resources for Experimentation

To truly understand how a tokenizer works, it's best to try it

- **Tiktokenizer (OpenAI):** A visual tool that lets you paste text and see exactly how different models (like GPT-4 and GPT-3.5) break the text into tokens.
Link: <https://tiktokenizer.vercel.app/>
- **Hugging Face Tokenizer Playground:** A general-purpose tool to experiment with various tokenizer algorithms and settings.
Link: https://huggingface.co/docs/transformers/tokenizer_summary

Tutorial on Tokenization

The [Let's Build the GPT Tokenizer](#) video from Andrej Karpathy (2hr15) provides an in-depth, self-contained, code-based description of tokenization in general and BPE specifically.